

Verifier Quality & Transformation Guide

0. What this document is

A trainer-facing guide to writing and fixing verifiers: the seven ways verifiers most commonly break, how to recognize each one in a real manifest, what the fix looks like, and a ready-to-use prompt for transforming a broken verifier suite into a fair one.

 [verifier_transform.zip](#) — a script (`apply_transform.py`) plus its prompt templates that automate rewriting a task's verifiers against these same rules, validate what comes back, and write it to a draft folder for review. Use this instead of hand-running a prompt yourself.

1. What a high-quality verifier establishes

Before touching any verifier, know what it has to prove. A high-quality verifier establishes that:

1. the task is clearly defined;
2. the verifier checks what the task actually requires;
3. valid solutions are accepted regardless of harmless presentation differences;
4. substantive errors are rejected;
5. the verifier is stable and reproducible;
6. the environment and harness make the task genuinely executable;
7. rewards reflect task performance rather than infrastructure or evaluator artifacts; and
8. an agent cannot obtain a high reward without genuinely completing the intended work.

Every fix below exists to restore one of these eight properties.

2. The failure taxonomy — 7 ways verifiers break

Nearly all verifier feedback maps to one of these, ordered by how often it shows up and how much damage it does:

#	Failure mode	Typical share	One-line symptom
1	Brittle regex / exact-string grading	~50%	Correct answer rejected over word order, Markdown, a synonym, hyphenation, or numeric notation.
2	Instruction–verifier mismatch	~25%	Grades the final chat message (or wrong file) when the instruction sent output elsewhere.
3	Infrastructure / harness failures scored as model results	~15%	API error, verifier timeout, or harness quirk → run scored 0.0.
4	Undocumented policy assumptions	~7%	Verifier asserts a rule the instruction never states.
5	All-or-nothing aggregation	~3%	63/64 checks pass → total reward 0.0.
6	LLM judge missed the trajectory	recurring	Judge claims work didn't happen when the trajectory shows it did.
7	Dead / hollow deterministic checks	recurring	Empty or disabled buckets silently renormalize onto one rubric item.

Failure mode 1 alone accounts for roughly half of all verifier defects — start there on every task.

3. Failure mode 1 — Brittle regex and exact-string grading (*the dominant problem*)

Typical false negatives you'll see:

- A memo abbreviates "3 August" as "3 Aug," or writes counts in a different word order — rejected despite being correct.
- A correct paraphrase fails a narrow synonym whitelist ("do not change capital interest" vs. the one accepted phrase).
- Negation-blind matching: "this is NOT a gap" still matches the bare word "gap" and is scored as a positive finding.
- Plurals / hyphenation break exact matches: "savings plan" passes but "savings plans" and "savings-plan coverage" fail.
- Weak alternation: a pattern collapses to its weakest branch, so a wrong answer slips through (or a good one is false-failed).

Anti-pattern — fragile, exact prose, unanchored, negation-blind:

```
JSON
{
  "path": "$.text",
  "comparison": "regex_match",
  "expected": "largest-remainder method"    // rejects "largest remainder"
}
{
  "comparison": "contains",
  "expected": "gap"                        // matches "not a gap"
}
{
  "comparison": "regex_match",
  "expected": "cap|26 weeks|uncapped"      // collapses to just "cap"
}
```

Better — grade the fact structurally from `results.json`:

```
JSON
{
  "source": { "type": "file", "file": { "type": "json",
    "command": "read_file",
    "arguments": { "path": "results.json" } } },
  "assertion": { "type": "deterministic", "expected": 3,
```

```
"deterministic": { "path": "$.invalid_codes",  
                  "comparison": "equals" } }  
}
```

If you must match prose, anchor and lengthen the stem:

```
None  
"expected": "\\backnowledg\\w*"           // word-bounded, meaningful stem
```

For anything semantic, use an LLM judge — not a regex.

Rules:

- Grade the fact structurally, not the prose. If it's a number or exact field, assert **equals** on a JSON value — never grep the memo for it.
- When you must match prose, normalize first (strip Markdown, normalize hyphens, whitespace, dates, numeric notation), then accept an equivalence set — not one literal.
- Prefer an LLM judge for explanations and justifications. If a human would accept a paraphrase, a regex must not be the grader.
- Anchor and lengthen every regex you keep. Use **\\b** and a stem long enough to mean something. Require two concepts to co-occur for stronger checks.
- Judge the weakest branch of any alternation — that is the strength of the whole pattern.
- Make prose matching negation-aware; evaluate each clause, not a whole section scanned for isolated failure words.
- For **contains**, use an identifier, not an English word ("**ESC-105**", "**2026-06-30**" — not "**interest**", "**term**", "**gap**").

4. Failure mode 2 — Instruction–verifier mismatch

The verifier grades a different deliverable than the instruction asked for — most often the final chat message when the instruction sent the write-up to a file, or an exact label/title the instruction never specified.

Mismatch — instruction says "write the report to **report.md**," but the check grades the final chat message:

JSON

```
"source": { "target": "final_answer" } // wrong target
```

Fix — grade the file the instruction named:

JSON

```
"source": { "type": "file", "file": {  
  "type": "md", "command": "extract_text",  
  "arguments": { "path": "report.md" } } }
```

Or change the instruction to explicitly require the figures in chat. Pick one — don't let the two drift.

Rules:

- Grade the deliverable where the instruction put it (file sources are supported).
- Do not require literal titles/labels the instruction never specified; accept the names the source data uses.
- For every check, quote the exact instruction sentence that creates the requirement and confirm the grading target matches.

5. Failure mode 4 — Undocumented policy assumptions

The verifier must only assert what a capable agent could infer from the instruction and environment alone. A check that encodes a hidden rule fails agents who followed the written policy literally.

Rules:

- Every verifier requirement must trace to an instruction sentence. If it doesn't, add it to the instruction or delete the check.
 - Resolve ambiguity before it reaches the grader (e.g. boolean vs. count, the scope of a notes section). Make the instruction unambiguous or accept both readings.
 - Don't re-check in prose what a structured field already proves (e.g. requiring the digit "3" near a keyword in the memo when `results.json` already has the count).
-

6. Failure mode 5 — All-or-nothing aggregation

Reward is 1.0 only if every check passes, so a single defective check zeroes a materially correct run. Most "63/64 passed, reward 0.0" cases are a defective check, not an incomplete answer.

Rules:

- Fix the underlying brittle/mismatched checks first — that removes most of these failures.
 - Where supported, weight and aggregate so one non-critical check cannot zero an otherwise correct run; reserve hard gates for load-bearing facts.
 - Keep the behavioral gate small and structured (a few derived figures in `results.json`); make secondary prose/format checks non-fatal or semantic.
-

7. Failure mode 6 — LLM judge missed the trajectory

The judge must read the actual trajectory and cite observable evidence.

Rules:

- Judge scoped/search calls and value reuse, not only direct reads or literal hardcoding.
 - Never treat null/missing metadata as evidence of absence — grade whether the target record was retrieved, or fix the metadata.
 - Accept unambiguous aliases, or score identity resolution as a separate, explicit item.
 - Confirm the judge rationale cites the trajectory; run repeats to ensure verdicts don't flip on identical deliverables.
-

8. Failure mode 7 — Dead / hollow deterministic checks

A check that never runs, or that grades a file the model writes about itself, measures nothing while inflating the apparent rigor of the manifest.

Hollow — grading a transcript the model authors:

```
JSON
{
  "source": { "file": { "type": "text",
    "arguments": { "path": "pytest_out.txt" } } },
```

```

"assertion": { "expected": "9 passed",
               "deterministic": { "path": "$.text",
                                   "comparison": "regex_match" } }
}
// The model can just TYPE "9 passed". And "99 passed" also matches.

```

Instead — execute the real suite against the model's code, and grade the output it produces (`results.json`), not a self-report.

Rules:

- Delete or fill empty weighted buckets before shipping; empty SQL/state buckets silently renormalize weight onto one rubric item.
- Execute code, don't grep it. Ship a real test suite and run it against the model's code at grading time.
- Never grade a file the model authors about its own run — reduce those to an `exists` check at most.

9. Decision framework: which grader to use

The requirement is...	Use	Never use
A derived number / exact field	<code>equals</code> (<code>approx_equals</code> for floats) on a JSON value	a regex over the memo
A money / ratio figure	<code>approx_equals</code> with a tolerance	exact <code>equals</code> on a float
A record exists in a CSV	row-anchored, quote-tolerant regex	the ID matched anywhere in the blob
A concept must appear in prose	anchored word/phrase regex with a meaningful stem	bare <code>contains</code> on an English word
An explanation / justification is correct	LLM judge citing evidence	a synonym-whitelist regex
A behavior is implemented in code	execute a real test suite	grep the source

The requirement is...	Use	Never use
A record must NOT be flagged	<code>not_regex_match</code> on the delimited field	—
Identity / alias resolution	explicit item accepting valid aliases	exact-name substring

When you're staring at a check and unsure how to grade it, find its row here first.

10. Reward-hacking review

Adversarially review every high-weight verifier. For each one, ask: **what's the cheapest way to pass without doing the work?** Confirm the agent cannot:

- Hardcode the expected answer
 - Read golden data
 - Write directly to verifier state
 - Fabricate a placeholder or transcript artifact
 - Spoof a required string
 - Satisfy a rubric while skipping a required side effect
 - Exploit stale state
 - Exploit a path mismatch
 - Manipulate the judge via prompt injection in an input
 - Get credit without retrieving the underlying evidence
-

11. Verifier distribution across evidence sources

Spread the suite across four sources so no single weak surface decides the score:

- **Trajectory** — proves the work happened (right records retrieved, side effects issued).
- **Deliverable** — the files the instruction named; primary content lives here.
- **DB / environment state** — proves side effects actually landed; strongest anti-spoof.
- **Final answer** — grade only when the instruction asks for content in the chat reply.

Aim for a small structured gate on the deliverable or DB state, backed by trajectory and secondary checks. Don't concentrate all weight on one source.

12. The three-question test — before any check ships

Every verifier check must answer all three, or it doesn't ship:

A. What requirement is this checking? The reviewer must be able to point to the relevant instruction or task specification.

B. Why is this check sufficient? Explain why passing the check establishes the requirement.

C. Why is this check appropriately permissive? Explain why a valid alternative solution would not be rejected merely because it uses different wording, ordering, formatting, aliases, or an equivalent implementation.

A verifier check without a clear answer to all three questions should not ship.

13. How to actually transform a verifier suite

1. **Read the manifest against the instruction**, checking each of the 7 failure modes above in order of §2's frequency — start with brittle regex, then mismatch, then the rest.
2. **Run the transform script instead of hand-writing a prompt** — [verifier_transform.zip](#).

None

```
# Dry run - writes a draft, touches nothing live
python verifier_transform\apply_transform.py --task "<task path>"

# Review transformed_tasks/<task_name>/, then apply for real (keeps a .bak)
python verifier_transform\apply_transform.py --task "<task path>" --write
```

It packs the task's instruction, README, solution, existing verifiers, and expected outputs into the transformation prompt, sends it to the same GLM gateway the QC judges use, and validates what comes back **before writing anything**:

- every regex compiles and every `deterministic.path` is a valid JSONPath;
- any rubric-type verifier carries a top-level `config.models` (odd count) and `config.temperature`;
- for connector tasks, no weighted `scoring.weights` bucket is left with an empty verifier list;

- `coverage` only references verifier names that actually exist.

A dry run always writes the transformed file plus a `transform_report.json` (coverage map, changes log, `instruction_gaps`) to `transformed_tasks/<task_name>/` without touching the live task. Only `--write` overwrites the live `verifier.json/manifest.json` — and only if validation passed. Use `--lean` on large connector tasks that would otherwise time out, or `--render-only` to inspect the filled prompt without calling the model at all. `--folder <dir>` runs the same thing concurrently over every task under a directory.

3. **Never accept the output blindly, even when validation passes.** The validator checks *shape* — regex compiles, config is present, names resolve — not judgment. It will not catch a weakened correctness check or an invented requirement. Review every changed check against the decision framework (§9) and the three-question test (§12) before it ships.
4. **Re-run the Oracle** after any verifier change — it must still score exactly 1.0.
5. **Run the reward-hacking review (§10)** on every high-weight check you touched.

What the transformation prompt actually asks for

The script ships two versions of the prompt: `TRANSFORM_PROMPT.md` (the full version, used by default — includes the exact JSON output contract and the `verifier.json/manifest.json` schema) and `TRANSFORM_PROMPT_LEAN.md` (a condensed version for large tasks, used with `--lean`). Both instruct the model to fix the same five defects covered in §3–§8 of this guide, spread coverage across the same four evidence sources as §11, and return one JSON object containing the rewritten `verifier.json`, a `coverage` map back to quoted instruction sentences, a `changes` log tagged by defect type, and any `instruction_gaps` it found but couldn't fix. Both carry the same hard constraints: never weaken a real correctness check, never invent a requirement, keep the given solution passing, preserve `task_id`.

 **Don't accept the changes without reviewing them first.** The script's validation only checks that the output is well-formed — not that it's fair. Read step 3 above.

Sources

- [Verifier Quality Guide](#) — the failure taxonomy, decision framework, reward-hacking checklist, and transformation prompt this guide is built from.

- [verifier_transform.zip](#) — `apply_transform.py` and both prompt templates (`TRANSFORM_PROMPT.md`, `TRANSFORM_PROMPT_LEAN.md`) for transforming a task's verifiers.